

1. What is Indexing? How does Indexing work?

A **Database Index** is a data structure that helps retrieve data quickly from a database table. It improves search speed but requires extra storage space and makes insert, update, and delete operations slightly slower.

Book Example:

An index in a database works like the index at the back of a textbook. Instead of reading the whole book, we find the topic in the index and directly go to the required page.

How does Indexing work

Indexing works like the **index of a book**. Instead of searching every page, we first look at the index, find the page number, and then directly go to the required page.

Similarly, in a database, an **index stores search-key values and pointers to the corresponding records**. When a user searches for data, the database first searches the index and then directly accesses the required record. This reduces the number of records that need to be scanned and makes data retrieval much faster.

2. What is Primary, Secondary Index and Multilevel Index? Difference between Primary, Secondary Index and Multilevel Index?

Primary Index:

Primary Index is an index in which the data file is organized (sorted) according to the search key.

Primary Index is the data structure of an indexing in which the data file is organized (sorted) according to the search key.

Example:

In the **Dense Index** and **Sparse Index** figures, records are stored by branch name and indexing also created by branch names (**Brighton, Downtown, Mianus, Perryridge, Redwood, Round Hill**). Therefore, the index on branch name is a **Primary Index**.

❑ Dense Index

Brighton		→	A-217	Brighton	750		↓
Downtown		→	A-101	Downtown	500		↓
Mianus		→	A-110	Downtown	600		↓
Perryridge		→	A-215	Mianus	700		↓
Redwood		→	A-102	Perryridge	400		↓
Round Hill		→	A-201	Perryridge	900		↓
		→	A-218	Perryridge	700		↓
		→	A-222	Redwood	700		↓
		→	A-305	Round Hill	350		↓

❑ Sparse Index

Brighton		→	A-217	Brighton	750		↓
Mianus		→	A-101	Downtown	500		↓
Redwood		→	A-110	Downtown	600		↓
		→	A-215	Mianus	700		↓
		→	A-102	Perryridge	400		↓
		→	A-201	Perryridge	900		↓
		→	A-218	Perryridge	700		↓
		→	A-222	Redwood	700		↓
		→	A-305	Round Hill	350		↓

Secondary Index

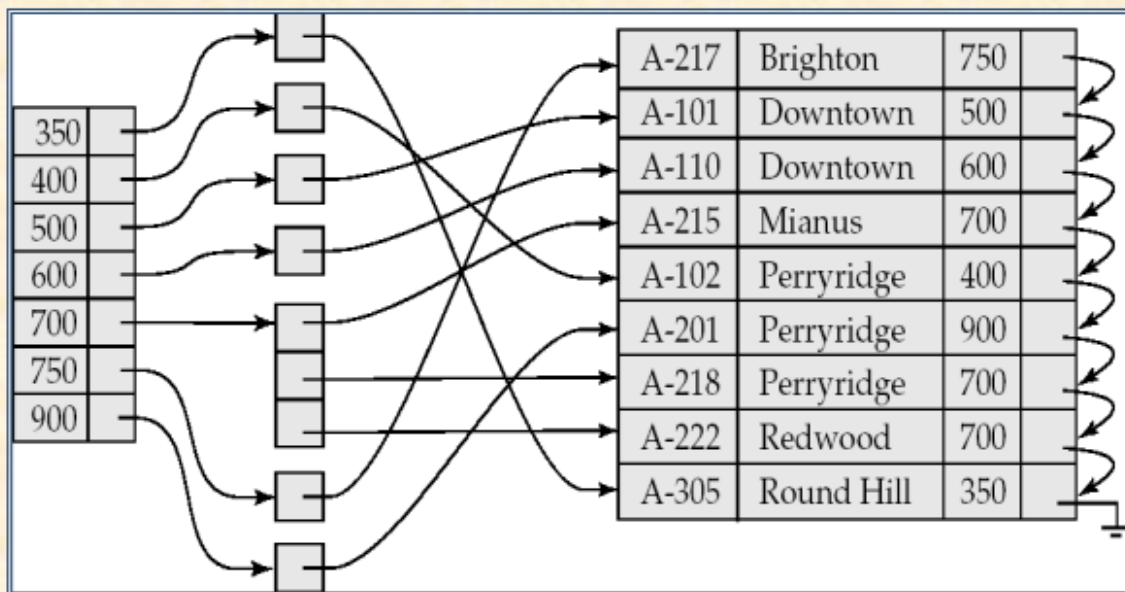
A **Secondary Index** is an index whose search key does not determine the physical order of records.

Figure Example

In the **Secondary Index** figure, records are stored by branch name, but the index is created on balance values (350, 400, 500, 600, 700, 750, 900).

So the balance field creates a **Secondary Index**.

❑ Secondary Index



Multilevel Index:

When an index becomes very large, another index is created on that index. This is called a **Multilevel Index**.

Figure Example

In the **Multilevel Index** figure:

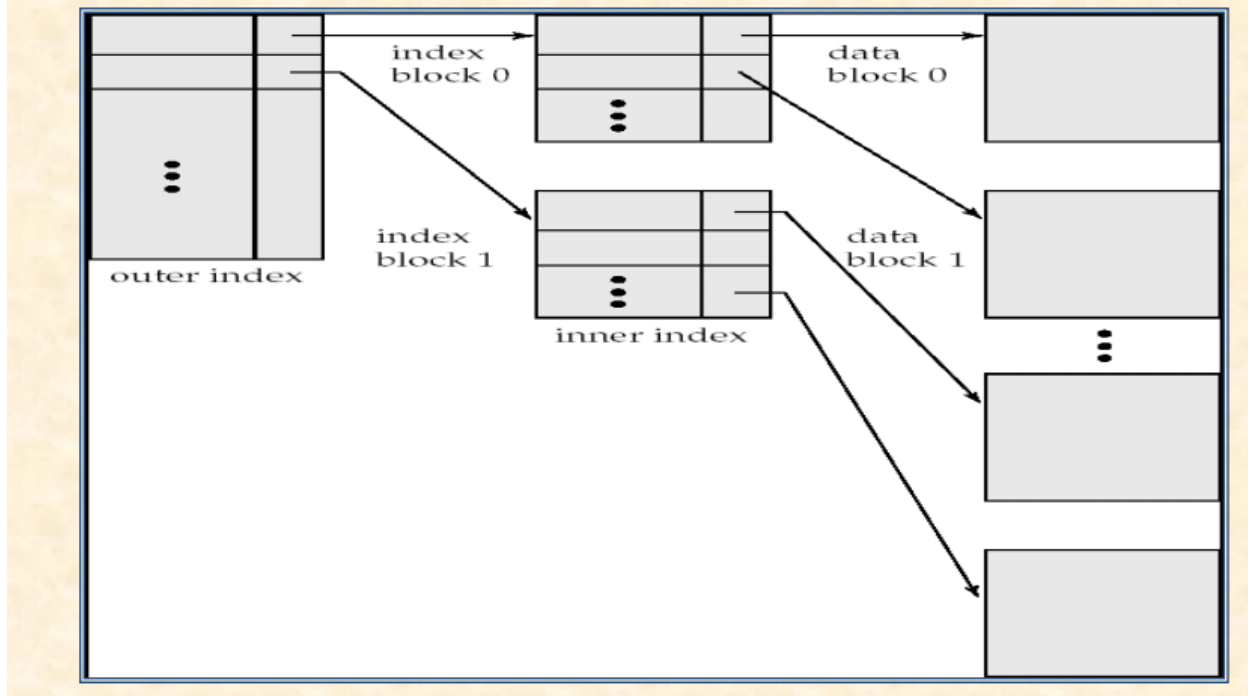
- Outer Index → points to Inner Index
- Inner Index → points to Data Blocks

So searching becomes:

Outer Index → Inner Index → Data Record

which is much faster.

❑ Multi-Level Indices: An Example



Difference Between Primary, Secondary and Multilevel Index:

Feature	Primary Index	Secondary Index	Multilevel Index
Definition	Index based on the field by which records are physically stored	Index based on a field that does not determine the physical order of records	An index created on another index
File Ordering	Data file is sorted by the search key	Data file is not sorted by the search key	Works on multiple levels of indices
Purpose	Fast access to ordered records	Provides an additional access path	Improves search efficiency for very large indices
Number per File	Usually one	Multiple can exist	Multiple levels can exist

Feature	Primary Index	Secondary Index	Multilevel Index
Storage Space	Less	More	More than Primary and Secondary
Search Speed	Fast	Fast, but slower than Primary	Fastest for very large files
Example	Records stored by Student ID and indexed on Student ID	Records stored by Student ID but indexed on Student Name	Outer Index → Inner Index → Data Record

3. What is Dense and Sparse index? Difference between them?

Dense Index

A Dense Index contains an index entry for **every search-key value** in the data file.

Based on the Figure

In the Dense Index figure, every branch name has an index entry:

- Brighton
- Downtown
- Mianus
- Perryridge
- Redwood
- Round Hill

Each entry directly points to the corresponding record.

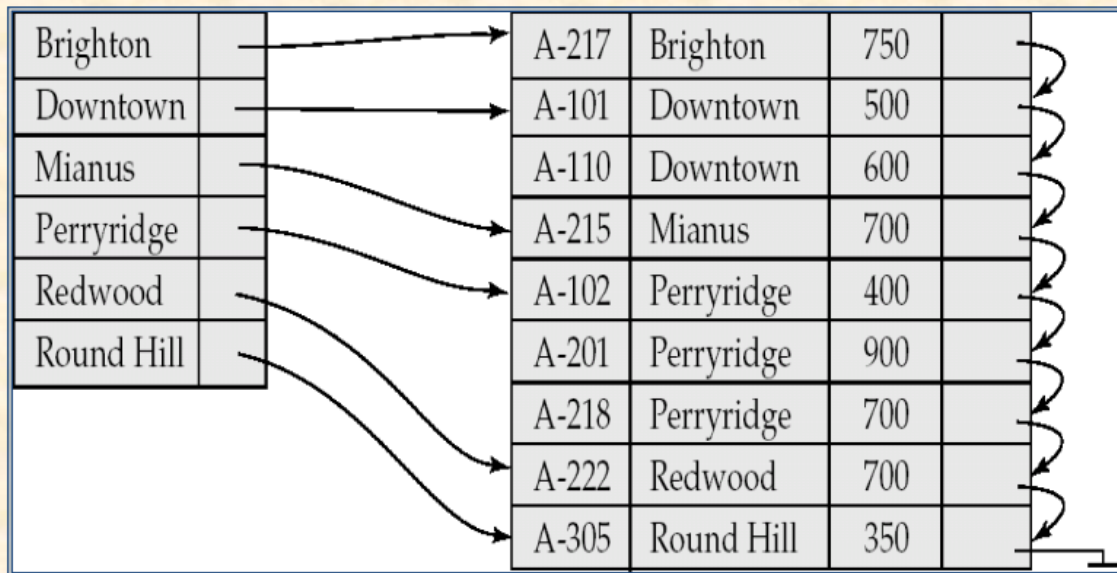
Easy Bangla

Dense Index-এ Data File-এর প্রতিটি Search Key-এর জন্য Index Entry থাকে।

ছবিতে দেখা যাচ্ছে প্রতিটি Branch Name-এর জন্য আলাদা Entry আছে, তাই এটি Dense Index।

☞ **Every Record = One Index Entry**

❑ Dense Index



Sparse Index

A **Sparse Index** contains index entries for **only some search-key values**.

Based on the Figure

In the Sparse Index figure, only:

- Brighton
- Mianus
- Redwood

have index entries.

Downtown, Perryridge, and Round Hill-এর জন্য কোনো Entry নেই।

প্রথমে কাছের Entry খুঁজে বের করতে হয়, তারপর Data File scan করে কাঙ্ক্ষিত Record পাওয়া যায়।

Easy Bangla

Sparse Index-এ সব Search Key-এর জন্য Entry থাকে না, শুধুমাত্র কিছু Key-এর জন্য Entry থাকে।

ছবিতে শুধু Brighton, Mianus এবং Redwood আছে।

☞ **Some Records = Some Index Entries**

❑ Sparse Index

Brighton		A-217	Brighton	750	
Mianus		A-101	Downtown	500	
Redwood		A-110	Downtown	600	
		A-215	Mianus	700	
		A-102	Perryridge	400	
		A-201	Perryridge	900	
		A-218	Perryridge	700	
		A-222	Redwood	700	
		A-305	Round Hill	350	

Difference Between Dense and Sparse Index

Dense Index

Index entry for every search key

Faster searching

More storage space required

More maintenance needed

Directly points to the record file

Sparse Index

Index entry for only some search keys

Slower searching

Less storage space required

Less maintenance needed

May need additional searching in data

4. Types of Indices? Discuss Evaluation Factors?

Types of Indices

There are **two basic types of indices**:

1. Ordered Index

Definition:

An Ordered Index is the data structure of an index where the data file are organized according to the search key values.

Features:

- Records are arranged according to the search key.
- Provides fast access to records.
- Each index is associated with a specific search key.
- A file may have multiple ordered indices.

Example: Primary Index, Secondary Index, Dense Index, Sparse Index.

2. Hash Index

Definition:

A Hash Index uses a **Hash Function** to distribute records into different buckets.

Features:

- Data is stored in buckets.
- The bucket is determined by a hash function.
- Very fast for exact searches.

Search Key

A **Search Key** is an attribute or set of attributes used to look up records in a file is called a search key.

Example

- Student ID
- Account Number

- Employee ID

Evaluation Factors of Indexing

→ **Access Types:** Access types can include finding records with a specified attribute value and finding records whose attribute values fall in a specified range.

→ **Access Time:** The time it takes to find a particular data item, or set of items, using the technique in question.

→ **Insertion Time:** The time it takes to insert a new data item. This value includes the time it takes to find the correct place to insert the new data item, as well as the time it takes to update the index structure.

→ **Deletion time:** The time it takes to delete a data item. This value includes the time it takes to find the item to be deleted, as well as the time it takes to update the index structure.

→ **Space overhead:** The additional space occupied by an index structure. Provided that the amount of additional space is moderate, it is usually worthwhile to sacrifice the space to achieve improved performance.

Difference Between Ordered Index and Hash Index

Ordered Index	Hash Index
Stores keys in sorted order	Uses hash function
Good for range search	Good for exact search
Data is ordered	Data is stored in buckets
Comparatively slower	Very fast

5. What is Index and Hashing? Difference between them?

Index

An **Index** is a data structure that helps retrieve records quickly from a database table.

It stores search-key values along with pointers to the corresponding records.

Easy Bangla

Index হলো এমন একটি Data Structure যা Database থেকে দ্রুত Data খুঁজে পেতে সাহায্য করে।

এটি Search Key এবং Record-এর Pointer সংরক্ষণ করে।

Example:

বইয়ের শেষে থাকা Index দেখে দ্রুত নির্দিষ্ট পৃষ্ঠা খুঁজে পাওয়ার মতো।

Hashing

Hashing is a technique that uses a **Hash Function** to map a search-key value to a specific bucket where the record is stored.

Easy Bangla

Hashing হলো এমন একটি পদ্ধতি যেখানে Hash Function ব্যবহার করে Data-কে নির্দিষ্ট Bucket-এ সংরক্ষণ করা হয়।

ফলে খুব দ্রুত Data খুঁজে পাওয়া যায়।

Example:

Student ID → Hash Function → Bucket → Record

Difference Between Index and Hashing

Index	Hashing
Uses an index structure	Uses a hash function
Search keys are often stored in sorted order	Data stored in buckets
Supports exact search and range search	Best for exact search

Index

Slower than hashing for exact match

Suitable for ordered data

Hashing

Very fast for exact match

Suitable for direct access

6. What is B+ tree / B tree?

A **B+ Tree** is a balanced multilevel index structure used in databases for fast searching, insertion, and deletion of records.

In a B+ Tree, **all actual data records are stored in the leaf nodes**, while non-leaf nodes contain only search keys and pointers.

Easy Bangla

B+ Tree হলো একটি Balanced Multilevel Index Structure যা Database-এ দ্রুত Search, Insert এবং Delete করার জন্য ব্যবহৃত হয়।

এখানে সব **Data Record Leaf Node-এ থাকে**, আর Non-Leaf Node-এ শুধু Search Key ও Pointer থাকে।

Key Features

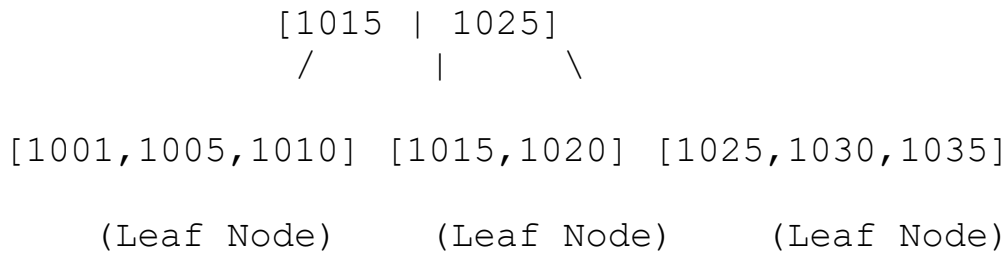
- Balanced Tree
- All leaf nodes are at the same level
- Supports fast searching
- Widely used in DBMS

Example:

ধরো একটি **Bank Database-এ** Account Number অনুযায়ী তথ্য সংরক্ষণ করা আছে:

1001, 1005, 1010, 1015, 1020, 1025, 1030, 1035

B+ Tree দেখতে এমন হতে পারে:



কী হচ্ছে এখানে?

- **Root Node (Non-Leaf)** = [1015 | 1025]
 - এখানে শুধু Search Key আছে।
 - Actual Data নেই।
- **Leaf Nodes**
 - সব Account Number এখানে সংরক্ষিত।
 - অর্থাৎ Actual Records Leaf Node-এ থাকে।

Search Example

ধরো Account Number **1030** খুঁজতে হবে।

Step 1: Root Node দেখো

[1015 | 1025]

1030 > 1025

তাই ডান পাশের Pointer Follow করব।

Step 2: ডান পাশের Leaf Node-এ যাব

[1025, 1030, 1035]

এখানে 1030 পাওয়া গেল।

✓ খুব দ্রুত Search সম্পন্ন হলো।

Explain node structure of B+ tree / B tree?

Node structure of a B+ Tree Node

According to your slide:

A node contains:

- Search Keys ($K_1, K_2, K_3 \dots$)
- Pointers ($P_1, P_2, P_3 \dots$)

Example

$P_1 \mid K_1 \mid P_2 \mid K_2 \mid P_3 \mid K_3 \mid P_4$

Rules

Non-Leaf Node

- Contains only keys and pointers
- Maximum = n pointers
- Minimum = $\lceil n/2 \rceil$ pointers

Leaf Node

- Contains actual data records
- Maximum = $n-1$ keys
- Minimum = $\lceil (n-1)/2 \rceil$ keys

Root Node

- If root is not leaf:
 - Minimum 2 children
- If root is leaf:
 - Can contain 0 to $n-1$ values

Fanout

The number of pointers in a node is called **Fanout**.

Example:

P1 K1 P2 K2 P3 K3 P4

There are 4 pointers.

→ Fanout = 4

Easy Bangla Answer

B+ Tree-এর প্রতিটি Node-এ:

- Search Key থাকে
- Pointer থাকে

Non-Leaf Node

শুধু Key ও Pointer রাখে।

Leaf Node

আসল Data Record রাখে।

Fanout

Node-এ যতগুলো Pointer থাকে তাকে Fanout বলে।

Exam Answer (5 Marks)

A B+ Tree node contains search-key values and pointers. Non-leaf nodes contain only keys and pointers, while leaf nodes contain actual records. A non-leaf node can have up to n pointers and at least $\lceil n/2 \rceil$ pointers. The number of pointers in a node is called fanout. All leaf nodes are maintained at the same

7. Create a B+ tree for order 10

Meaning of Order 10

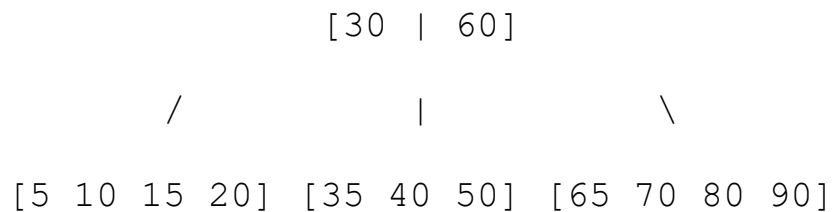
Order = 10 means:

- Maximum children = 10
- Maximum keys = 9

Because:

$$\begin{aligned}\text{Maximum Keys} &= \text{Order} - 1 \\ &= 10 - 1 \\ &= 9\end{aligned}$$

Example B+ Tree (Order 10)



Verification

Root Node:

- Keys = 2
- Children = 3

Leaf Nodes:

- Keys are less than or equal to 9 ✓

All leaf nodes are at the same level ✓

Therefore it is a valid B+ Tree of order 10.